Comprehensive Course: Large Language Models (LLMs) – From Theory to Production

Author: Bassem Ben Hamed
Date: February 2026
Mail: bassem.benhamed@enetcom.usf.tn
Target Audience: Academic Researchers, Data Scientists, Software Engineers, and Technical Managers.
Format: Mixed (Theoretical Lectures & Hands-on Labs)

Prerequisites

> [!IMPORTANT]
> Participants should have the following background before attending:

Required:

- Intermediate Python programming (functions, classes, libraries)
- Basic Machine Learning concepts (supervised learning, loss functions, gradient descent)
- Familiarity with Jupyter Notebooks

Recommended:

- Basic linear algebra (vectors, matrices, dot products)
- Prior experience with PyTorch or TensorFlow
- Familiarity with APIs (REST, JSON)

Learning Objectives

By the end of this training, participants will be able to:

1. **Understand** the architectural foundations of Transformers and LLMs.
2. **Master** prompt engineering techniques to steer model behavior effectively.
3. **Implement** Retrieval-Augmented Generation (RAG) systems for domain-specific tasks.
4. **Apply** fine-tuning strategies (SFT, RLHF, DPO) and evaluate model performance systematically.
5. **Implement** standardized LLM-tool integration using the Model Context Protocol (MCP).
6. **Design** and build autonomous AI agent systems with multi-agent orchestration.
7. **Deploy** optimized LLM solutions with proper MLOps practices and safety considerations.

Part 1: Natural Language Processing Fundamentals

Goal: Establish foundational understanding of NLP and the evolution of language modeling techniques.

1.1 Overview of Natural Language Processing

Theory: Introduction to NLP

- **What is Natural Language Processing?**
  - Definition and scope of NLP
  - Core challenges: Ambiguity, context, variability
  - Major NLP tasks: Classification, Named Entity Recognition, Machine Translation, Summarization, Question Answering
- **Historical Evolution of Language Models:**
  - Rule-based systems and Expert Systems
  - Statistical NLP: N-grams, Hidden Markov Models (HMMs)
  - Bag-of-Words, TF-IDF
  - Word embeddings: Word2Vec, GloVe, FastText

1.2 Text Representation and Evaluation

Theory: Representing and Measuring Language

- **Text Representation:**
  - Tokenization fundamentals (word-level, character-level)
  - Vector space models
  - Distributed representations and semantic similarity
- **Evaluation Metrics in NLP:**
  - Accuracy, Precision, Recall, F1-Score
  - Perplexity for language models
  - BLEU, ROUGE for generation tasks

Lab 1: Classical NLP Techniques

- **Tools:** Python, NLTK, scikit-learn, Gensim.
- **Tasks:**
  - Text preprocessing and tokenization
  - Building a TF-IDF-based text classifier
  - Training Word2Vec embeddings on a corpus
  - Exploring word similarities and analogies in vector space
  - Comparing classical methods with modern approaches

Part 2: From Recurrent Networks to Transformers

Goal: Understand the architectural evolution from sequential models to attention-based architectures.

2.1 Recurrent Neural Networks (RNN) and Their Limitations

Theory: Sequential Models

- **Introduction to RNNs:**
  - Architecture and hidden states
  - Forward propagation through time
  - Applications: Language modeling, sequence prediction
- **Vanishing and Exploding Gradients:**
  - Challenges in training deep RNNs
  - Long-term dependency problem
- **Advanced RNN Architectures:**
  - **LSTM (Long Short-Term Memory):** Gates mechanism (forget, input, output)
  - **GRU (Gated Recurrent Units):** Simplified gating
  - Bidirectional RNNs (Bi-LSTM, Bi-GRU)
- **RNN Use Cases:**
  - **Text Classification:** Sentiment analysis, spam detection
  - **Sequence-to-Sequence (Seq2Seq):** Machine translation, text summarization
  - Encoder-Decoder architecture with attention mechanism (Bahdanau et al. 2015)

Lab 2: Building RNN-based Models

- **Tools:** PyTorch/TensorFlow, Hugging Face Datasets.
- **Tasks:**
  - Implementing a text classification model with LSTM
  - Building a Seq2Seq model for simple translation task
  - Visualizing hidden states and gradient flow
  - Understanding the limitations of RNNs on long sequences

2.2 The Transformer Revolution

Theory: Attention Is All You Need

- **Limitations of RNNs:**
  - Sequential computation bottleneck
  - Difficulty parallelizing training

    - Limited context window
- **The Transformer Architecture (Vaswani et al. 2017):**
    - **Self-Attention Mechanism:** Query, Key, Value matrices
    - **Multi-Head Attention:** Parallel attention heads
    - **Position-wise Feed-Forward Networks**
    - **Positional Encodings:** Sinusoidal, Learned
    - Layer Normalization and Residual Connections
- **Transformer Variants:**
    - **Encoder-only:** BERT, RoBERTa (understanding/classification tasks)
    - **Decoder-only:** GPT family (generation tasks)
    - **Encoder-Decoder:** T5, BART (sequence-to-sequence tasks)
- **Advantages over RNNs:**
    - Parallel processing of sequences
    - Better handling of long-range dependencies
    - More efficient training on GPUs

Theory: Tokenization and Embeddings

- **Modern Tokenization:**
    - Subword tokenization: BPE (Byte Pair Encoding)
    - WordPiece, SentencePiece, Unigram LM
    - Tiktoken (OpenAI's tokenizer)
    - Trade-offs: vocabulary size vs. sequence length
- **Advanced Positional Encodings:**
    - RoPE (Rotary Position Embedding)
    - ALiBi (Attention with Linear Biases)
    - Impact on context length extension

Lab 3: Transformer for Classification and Seq2Seq

- **Tools:** PyTorch, Hugging Face Transformers.
- **Tasks:**
    - Fine-tuning BERT for text classification (sentiment analysis)
    - Using T5 for sequence-to-sequence tasks (summarization, translation)
    - Comparing Transformer vs. LSTM performance on same tasks
    - Visualizing attention weights and heads
    - Experimenting with different tokenizers and their impact

---

Part 3: From Transformers to Large Language Models

Goal: Understand how transformers evolved into modern LLMs and their capabilities.

3.1 The Emergence of Large Language Models

Theory: Scaling Transformers

- **The GPT Family Evolution:**
  - GPT-1: Unsupervised pre-training + supervised fine-tuning
  - GPT-2: Scaling and zero-shot learning
  - GPT-3: Few-shot learning and in-context learning
  - GPT-4, GPT-4o: Multimodal and advanced reasoning
- **Open-Source LLM Ecosystem:**
  - LLaMA family (Meta): LLaMA 1, 2, 3, 3.1, 3.2
  - Mistral, Mixtral (Mistral AI)
  - Phi family (Microsoft)
  - Gemma (Google), Qwen (Alibaba)
- **Scaling Laws:**
  - Relationship between compute, data size, and parameters (Kaplan et al., Chinchilla)
  - Optimal model size vs. training tokens
  - Emergent abilities at scale
- **Multimodal LLMs:**
  - Vision-Language Models: GPT-4V/4o, LLaVA, Claude Vision
  - Audio and speech integration (Whisper, GPT-4o audio)
  - Unified multimodal architectures
  - Applications: image understanding, document analysis, video comprehension

Theory: Pre-training and Autoregressive Generation

- **Pre-training Pipeline:**
  - Self-supervised learning on massive datasets
  - **Training Data Sources:** Common Crawl, The Pile, RedPajama, RefinedWeb, FineWeb
  - Data cleaning, deduplication, and filtering
- **Next-Token Prediction:**
  - Autoregressive generation
  - Cross-entropy loss and perplexity
- **Decoding Strategies:**

    - Greedy decoding vs. sampling
    - Temperature, Top-k, Top-p (nucleus sampling)
    - Repetition penalty and frequency penalty
- **Context Window:**
    - Sequence modeling limitations
    - Techniques for extending context (RoPE scaling, attention modifications)
    - Long-context models: GPT-4 Turbo, Claude, Gemini Pro

Lab 4: Interacting with Foundation Models

- **Tools:** Python, Hugging Face Transformers, OpenAI API, Anthropic API.
- **Tasks:**
    - Loading and comparing open-source models (Llama 3, Phi-3, Mistral)
    - Generating text with different decoding strategies
    - Analyzing the impact of temperature and sampling parameters
    - Visualizing token probabilities and generation process
    - Comparing tokenizers across different model families
    - Testing zero-shot and few-shot capabilities

3.2 Prompt Engineering, In-Context Learning & Function Calling

Theory: Steering LLM Behavior

- **Prompt Engineering Fundamentals:**
    - Zero-shot and few-shot prompting
    - System prompts and role definition
    - Structured outputs (JSON mode)
    - Prompt templates and best practices
- **Chain-of-Thought (CoT) Reasoning (Wei et al. 2022):**
    - Step-by-step reasoning
    - Zero-shot vs. few-shot CoT
    - Self-Consistency: Sampling multiple reasoning chains
- **Advanced Prompting Techniques:**
    - Tree-of-Thoughts (ToT): Exploring multiple reasoning paths
    - Graph-of-Thoughts: Structured reasoning graphs
    - Prompt chaining and decomposition
- **Function Calling:**
    - JSON schema tool definitions

- Single and parallel tool calls
- Function calling across providers (OpenAI, Anthropic, open-source)
- Structured output parsing and validation
- Error handling and fallback strategies

Lab 5: Prompt Engineering & Function Calling

- **Tools:** OpenAI API, Anthropic API, Hugging Face Transformers.
- **Tasks:**
  - Implementing zero-shot, few-shot, and CoT prompting strategies
  - Comparing prompt techniques across different models
  - Building function calling workflows with tool definitions
  - Implementing structured output extraction (JSON mode)
  - Evaluating prompt effectiveness on benchmark tasks

Part 4: Retrieval-Augmented Generation (RAG)

Goal: Learn to ground LLMs with external knowledge sources to reduce hallucinations and enable domain-specific applications.

4.1 Foundations of RAG

Theory: Beyond Parametric Knowledge

- **Limitations of Pure LLMs:**
  - Knowledge cutoff dates
  - Hallucinations and factual errors
  - Inability to access private/proprietary data
  - Static knowledge base
- **RAG Architecture (Lewis et al. 2020):**
  - Retriever + Generator pipeline
  - Dense retrieval vs. sparse retrieval
  - End-to-end vs. modular approaches
- **Prompt Strategies for RAG** (building on Part 3.2):
  - Context injection and grounding techniques
  - RAG-specific prompt templates
  - Handling retrieval failures in prompts
  - Citation and source attribution in responses

Theory: Embeddings and Semantic Search

- **Embedding Models:**
  - OpenAI Ada (text-embedding-ada-002, text-embedding-3-small/large)
  - Sentence-Transformers (all-MiniLM, all-mpnet)
  - Cohere Embed, Voyage AI
  - Multilingual embeddings
- **Vector Similarity Metrics:**
  - Cosine similarity, Euclidean distance, dot product
  - Trade-offs between different metrics
- **Vector Databases:**
  - HNSW (Hierarchical Navigable Small World) indexing
  - Popular solutions: Chroma, Pinecone, Weaviate, Qdrant, Milvus, FAISS
  - Metadata filtering and hybrid search

4.2 Advanced RAG Techniques

Theory: Chunking, Retrieval, and Reranking

- **Document Chunking Strategies:**
  - Fixed-size chunking (token-based, character-based)
  - Semantic chunking (sentence, paragraph)
  - Recursive splitting with overlap
  - Document-specific strategies (Markdown, PDF, code)
- **Hybrid Search:**
  - Combining keyword search (BM25, TF-IDF) and vector search
  - Reciprocal Rank Fusion (RRF)
- **Reranking:**
  - CrossEncoder models (Cohere Rerank, sentence-transformers)
  - LLM-based reranking
  - Score fusion techniques
- **Query Transformations:**
  - Query expansion and reformulation
  - Hypothetical Document Embeddings (HyDE)
  - Multi-query retrieval
- **GraphRAG and Knowledge Graphs:**
  - Entity extraction and relationship mapping
  - Knowledge graph construction from documents

    - Graph-based retrieval and traversal
    - Combining vector search with graph queries
- **Agentic RAG:**
    - Self-correcting RAG with reflection loops
    - Adaptive retrieval (deciding when to retrieve)
    - Multi-step retrieval and reasoning
    - Router-based RAG (choosing retrieval strategy dynamically)
- **RAG Evaluation:**
    - Evaluation frameworks: Ragas, ARES
    - Metrics: Context Precision, Context Recall, Faithfulness, Answer Relevancy
    - End-to-end vs. component-level evaluation
    - Building custom evaluation pipelines

Lab 6: Building Production RAG Systems

- **Tools:** LangChain, LlamaIndex, Vector Stores (Chroma, Weaviate), Ragas.
- **Tasks:**
    - Ingesting and processing multi-format documents (PDFs, Markdown, Web pages)
    - Implementing different chunking strategies and comparing performance
    - Building a "Chat with your Data" application
    - Implementing hybrid search with reranking
    - Adding metadata filtering and source attribution
    - Building a GraphRAG pipeline with entity extraction
    - Implementing agentic RAG with adaptive retrieval
    - Evaluating RAG systems with Ragas (Faithfulness, Relevancy, Precision, Recall)
    - Handling edge cases: retrieval failures, irrelevant context

---

Part 5: Fine-Tuning and Model Adaptation

Goal: Learn to customize pre-trained models for specific domains and tasks using modern efficient techniques.

5.1 Supervised Fine-Tuning (SFT)

Theory: Full Fine-Tuning vs. Efficient Methods

- **When to Fine-Tune vs. Prompt Engineering:**
    - Decision framework: task complexity, data availability, cost
    - RAG vs. Fine-tuning vs. Hybrid approaches

- **Supervised Fine-Tuning (SFT):**
  - Instruction tuning: Teaching models to follow commands
  - Dataset formats: Alpaca, ChatML, ShareGPT, Conversational
  - Dataset curation and quality assessment
  - Catastrophic forgetting and mitigation strategies
- **Full Fine-Tuning:**
  - Forward and backward propagation
  - Gradient accumulation and mixed precision training
  - Computational and memory requirements
  - When is full fine-tuning necessary?

5.2 Parameter-Efficient Fine-Tuning (PEFT)

Theory: Efficient Adaptation Techniques

- **Why PEFT?**
  - Memory and computational constraints
  - Faster training and iteration
  - Multiple task-specific adapters
- **LoRA (Low-Rank Adaptation) - Hu et al. 2021:**
  - Low-rank matrix decomposition
  - Trainable rank decomposition matrices
  - Target modules: query, key, value, output projections
  - Rank (r) and alpha hyperparameters
- **QLoRA (Quantized LoRA) - Dettmers et al. 2023:**
  - 4-bit quantization with NormalFloat (NF4)
  - Double quantization
  - Training on consumer hardware (single GPU)
- **Other PEFT Methods:**
  - **Adapters:** Bottleneck layers inserted into transformer
  - **Prefix Tuning:** Learning continuous task-specific vectors
  - **Prompt Tuning:** Soft prompts as trainable parameters
  - **IA3 (Infused Adapter by Inhibiting and Amplifying Inner Activations)**
- **Comparison of PEFT Methods:**
  - Parameter efficiency, performance, training speed
  - Use cases for each method

5.3 Alignment and Preference Optimization

Theory: Aligning Models with Human Preferences

- **The Alignment Problem:**
  - Helpfulness, Harmlessness, Honesty (HHH)
  - Instruction following vs. safety
- **Reinforcement Learning from Human Feedback (RLHF):**
  - Three-stage pipeline: SFT → Reward Model → PPO
  - Reward modeling from preference data
  - Proximal Policy Optimization (PPO)
  - Challenges: complexity, instability, computational cost
- **Direct Preference Optimization (DPO) - Rafailov et al. 2023:**
  - Simplified alternative to RLHF
  - Direct optimization on preference pairs
  - No separate reward model or RL training
- **Modern Alignment Methods:**
  - **ORPO (Odds Ratio Preference Optimization):** Single-stage training
  - **KTO (Kahneman-Tversky Optimization):** Binary feedback
  - **IPO, CPO:** Other preference optimization variants
- **Creating Preference Datasets:**
  - Human annotation vs. AI-generated preferences
  - Quality control and inter-annotator agreement

Lab 7: Fine-Tuning with LoRA and QLoRA

- **Tools:** PyTorch, Hugging Face (`transformers`, `peft`, `trl`), Google Colab/Kaggle (GPU).
- **Tasks:**
  - Preparing a custom instruction dataset (domain-specific)
  - Fine-tuning Llama 3 or Phi-3 with LoRA on classification/generation task
  - Experimenting with QLoRA for larger models on limited hardware
  - Comparing different LoRA ranks and target modules
  - Merging LoRA adapters and exporting the final model
  - Evaluating base model vs. fine-tuned model performance

Lab 8: Preference Optimization with DPO

- **Tools:** Hugging Face TRL (Transformer Reinforcement Learning), Axolotl.
- **Tasks:**

- Creating or using a preference dataset (e.g., Anthropic HH-RLHF)
- Training a model with DPO
- Comparing SFT-only vs. SFT+DPO outputs
- Evaluating alignment quality (MT-Bench, AlpacaEval)

5.4 Evaluation and Deployment

Theory: Measuring Success and Serving Models

- **Evaluation Challenges:**
  - Open-ended generation, subjectivity
  - Task-specific vs. general capabilities
- **Benchmark Suites:**
  - **Knowledge & Reasoning:** MMLU, HellaSwag, ARC, TruthfulQA
  - **Coding:** HumanEval, MBPP
  - **Instruction Following:** MT-Bench, AlpacaEval, Arena-Hard
  - **Safety:** ToxiGen, BOLD, BBQ (Bias Benchmark)
- **Custom Evaluation:**
  - BLEU, ROUGE, METEOR, BERTScore
  - LLM-as-a-Judge (using GPT-4, Claude for evaluation)
  - Human evaluation protocols
- **Inference Optimization:**
  - **Quantization:** GPTQ, AWQ, GGUF (llama.cpp), bitsandbytes
  - **Serving Frameworks:** vLLM, TGI (Text Generation Inference), Ollama, LMStudio
  - KV-Cache, Continuous Batching, PagedAttention
  - Speculative Decoding, Flash Attention
- **Model Cards and Documentation:**
  - Intended use, limitations, bias considerations
  - Training data, methodology, performance metrics

Lab 9: Model Evaluation and Deployment

- **Tools:** lm-evaluation-harness, vLLM, Ollama, Gradio.
- **Tasks:**
  - Running standardized benchmarks (MMLU, HumanEval)
  - Implementing LLM-as-a-Judge evaluation
  - Quantizing models with different methods (GPTQ, GGUF)
  - Deploying a model with vLLM or Ollama

- Building a demo interface with Gradio or Streamlit
- Load testing and performance profiling

Part 6: Model Context Protocol (MCP)

Goal: Understand and implement standardized tool integration for LLMs using the Model Context Protocol.

6.1 Introduction to Model Context Protocol

Theory: Standardizing LLM-Tool Integration

- **The Tool Integration Challenge:**
  - Proliferation of custom tool implementations
  - Lack of interoperability between frameworks
  - Security and permission management
  - Maintenance overhead
- **Model Context Protocol (MCP) - Anthropic:**
  - Open protocol for connecting LLMs to data sources and tools
  - Client-server architecture
  - Standardized communication protocol (JSON-RPC)
  - Security model and sandboxing
- **MCP Architecture:**
  - **MCP Hosts:** Applications (Claude Desktop, IDEs)
  - **MCP Clients:** Protocol implementers
  - **MCP Servers:** Tool/data providers
  - **Resources, Prompts, and Tools:** Core primitives

Theory: MCP Components and Capabilities

- **Core Primitives:**
  - **Resources:** Data sources (files, databases, APIs)
  - **Prompts:** Reusable prompt templates
  - **Tools:** Executable functions
- **Transport Mechanisms:**
  - Standard I/O (stdio)
  - Streamable HTTP (replacing legacy SSE transport)
  - Local vs. remote servers
- **Security and Permissions:**

    - Capability-based security
    - User consent and authorization
    - Sandboxing and isolation
- **MCP vs. Other Tool Frameworks:**
    - Function calling (OpenAI, Anthropic native)
    - LangChain tools
    - When to use MCP

6.2 Building MCP Servers and Clients

Theory: Implementing MCP

- **MCP Server Development:**
    - Server SDK (Python, TypeScript)
    - Implementing resource handlers
    - Defining tool schemas and handlers
    - Error handling and validation
- **MCP Client Integration:**
    - Connecting to MCP servers
    - Discovering available tools
    - Invoking tools and handling responses
- **Common MCP Server Patterns:**
    - File system access
    - Database queries
    - API integrations
    - Web scraping
    - Custom business logic

Lab 10: Building MCP Servers

- **Tools:** MCP Python SDK, Claude Desktop, VS Code.
- **Tasks:**
    - Creating a simple MCP server for file system operations
    - Implementing a database MCP server (SQLite/PostgreSQL)
    - Building a custom API integration MCP server (weather, stocks, etc.)
    - Connecting MCP servers to Claude Desktop
    - Testing and debugging MCP implementations
    - Security considerations and best practices

Lab 11: MCP in Production

- **Tools:** MCP SDK, Docker, FastAPI.
- **Tasks:**
  - Deploying MCP servers with Streamable HTTP transport
  - Implementing authentication and authorization
  - Building a multi-resource MCP server
  - Monitoring and logging MCP server operations
  - Creating reusable MCP server templates

Part 7: Agentic AI and Autonomous Systems

Goal: Build autonomous AI agents that can reason, plan, and execute multi-step tasks using tools and orchestration frameworks.

7.1 Foundations of AI Agents

Theory: From Chatbots to Autonomous Agents

- **What Makes an Agent?**
  - Perception → Reasoning → Action loop
  - Autonomy, reactivity, proactivity, social ability
  - Agent vs. chatbot vs. workflow automation
- **Core Concepts:**
  - **Tool/Function Calling:** OpenAI, Anthropic, open-source implementations
  - **ReAct Pattern (Yao et al. 2023):** Reasoning + Acting
  - **Planning Strategies:**
    - Task decomposition (tree-based, sequential)
    - Reflection and self-correction
    - Iterative refinement
- **Memory Systems:**
  - **Short-term:** Conversation context window
  - **Long-term:** Vector store, graph databases
  - **Episodic:** Task history and learned experiences
  - Memory compression and summarization
- **Agent Architectures:**
  - Single-agent systems
  - Multi-agent collaboration

- Hierarchical agents and delegation

Theory: Reasoning and Planning

- **Reasoning Techniques for Agents** (building on Part 3.2):
  - Applying CoT, ToT, and Self-Consistency in agent loops
  - Reflection and self-correction patterns
  - Iterative refinement with feedback
- **Planning Algorithms:**
  - Forward planning (STRIPS-like)
  - Backward chaining
  - Heuristic search
  - Monte Carlo Tree Search (MCTS) for planning

7.2 Agent Frameworks and Orchestration

Theory: LangChain and LangGraph

- **LangChain Agents:**
  - Agent executor architecture
  - Agent types: Zero-shot ReAct, Structured Chat, OpenAI Functions
  - Tools ecosystem: Search, Calculator, Code Interpreter, Custom tools
  - Callbacks and observability
  - Limitations of traditional agents
- **LangGraph (Stateful Agent Orchestration):**
  - Graph-based workflow orchestration
  - State management and persistence
  - Cycles, branching, and conditional edges
  - Human-in-the-loop patterns
  - Checkpointing and time travel
  - Streaming and real-time updates
- **Agent Evaluation:**
  - Success rate, efficiency, cost
  - AgentBench, WebArena, ToolBench
  - Debugging agent failures

Lab 12: Building Agents with LangGraph

- **Tools:** LangChain, LangGraph, Python.
- **Tasks:**

- Creating a research agent with web search and document analysis
- Implementing a coding agent with code execution
- Building a customer service agent with database access
- Adding memory and state persistence
- Implementing human-in-the-loop approval steps
- Debugging and optimizing agent performance

7.3 Multi-Agent Systems

Theory: Multi-Agent Orchestration

- **Why Multi-Agent Systems?**
  - Specialization and division of labor
  - Parallel task execution
  - Fault tolerance and redundancy
- **Communication Patterns:**
  - Direct messaging
  - Shared memory/blackboard
  - Publish-subscribe
  - Coordinator-worker pattern
- **Multi-Agent Frameworks:**
  - **CrewAI:**
    - Role-based agents with goals and backstories
    - Task delegation and collaboration
    - Sequential vs. hierarchical processes
  - **AutoGen (Microsoft):**
    - Conversational agents and group chat
    - Code execution and debugging agents
    - Teachability and learning from feedback
  - **Microsoft Semantic Kernel:**
    - Plugin-based architecture
    - Orchestration with planners
- **Native Agent SDKs:**
  - **Anthropic Agent SDK:** Claude-native agent building
  - **OpenAI Agents SDK:** GPT-native agent orchestration
  - Comparison: framework-based vs. native SDKs

    - When to use native vs. third-party frameworks
- **Swarm Intelligence:**
    - Emergent behavior from simple rules
    - Consensus mechanisms
    - Conflict resolution strategies

Lab 13: Multi-Agent Systems

- **Tools:** CrewAI, AutoGen, LangGraph (multi-agent).
- **Tasks:**
    - Building a multi-agent research team (Researcher + Writer + Critic)
    - Implementing a code review system with multiple agents
    - Creating a hierarchical agent organization
    - Handling agent disagreements and consensus building
    - Optimizing multi-agent communication costs

7.4 Agent Deployment and Production

Theory: Production-Grade Agent Systems

- **Deployment Considerations:**
    - Reliability and error handling
    - Rate limiting and cost management
    - Monitoring and observability
    - Security and sandboxing
- **No-Code/Low-Code Agent Platforms:**
    - **n8n:** Visual workflow automation with LLM integration
    - **Flowise:** LangChain visual builder
    - **Dify:** LLMOps platform for agent deployment
    - **Zapier AI / Make.com:** Integration platforms
    - When to use code vs. no-code
- **Agent Observability:**
    - LangSmith, LangFuse
    - Tracing and debugging
    - Performance metrics and analytics
- **Safety and Alignment for Agents:**
    - Tool use safety
    - Output validation

- Constitutional AI principles for agents

Lab 14: Production Agent Deployment

- **Tools:** n8n, LangSmith, Docker.
- **Tasks:**
  - Building a visual agent workflow with n8n
  - Deploying agents with proper error handling
  - Implementing observability with LangSmith
  - Setting up rate limiting and cost controls
  - Creating monitoring dashboards
  - Testing agent robustness and safety

Resources & References

Academic Papers

Foundational NLP & Word Embeddings:

- Mikolov et al. (2013) - *Efficient Estimation of Word Representations in Vector Space* (Word2Vec)
- Pennington et al. (2014) - *GloVe: Global Vectors for Word Representation*

Recurrent Networks & Sequence Models:

- Hochreiter & Schmidhuber (1997) - *Long Short-Term Memory* (LSTM)
- Cho et al. (2014) - *Learning Phrase Representations using RNN Encoder-Decoder* (GRU)
- Bahdanau et al. (2015) - *Neural Machine Translation by Jointly Learning to Align and Translate* (Attention)

Transformers & Foundation Models:

- Vaswani et al. (2017) - *Attention Is All You Need*
- Devlin et al. (2018) - *BERT: Pre-training of Deep Bidirectional Transformers*
- Radford et al. (2018) - *Improving Language Understanding by Generative Pre-Training* (GPT-1)
- Brown et al. (2020) - *Language Models are Few-Shot Learners* (GPT-3)
- Raffel et al. (2020) - *Exploring the Limits of Transfer Learning with T5*
- Touvron et al. (2023) - *LLaMA: Open and Efficient Foundation Language Models*
- Jiang et al. (2023) - *Mistral 7B*
- Kaplan et al. (2020) - *Scaling Laws for Neural Language Models*
- Hoffmann et al. (2022) - *Training Compute-Optimal Large Language Models* (Chinchilla)
- Liu et al. (2023) - *Visual Instruction Tuning* (LLaVA)

RAG & Retrieval:

- Lewis et al. (2020) - *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*
- Karpukhin et al. (2020) - *Dense Passage Retrieval for Open-Domain Question Answering*
- Edge et al. (2024) - *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*

Fine-Tuning & Adaptation:

- Hu et al. (2021) - *LoRA: Low-Rank Adaptation of Large Language Models*
- Dettmers et al. (2023) - *QLoRA: Efficient Finetuning of Quantized LLMs*
- Houlsby et al. (2019) - *Parameter-Efficient Transfer Learning for NLP* (Adapters)
- Li & Liang (2021) - *Prefix-Tuning: Optimizing Continuous Prompts*

Alignment & RLHF:

- Christiano et al. (2017) - *Deep Reinforcement Learning from Human Preferences*
- Ouyang et al. (2022) - *Training Language Models to Follow Instructions with Human Feedback* (InstructGPT)
- Rafailov et al. (2023) - *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*
- Bai et al. (2022) - *Constitutional AI: Harmlessness from AI Feedback*

Agents & Reasoning:

- Wei et al. (2022) - *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*
- Yao et al. (2023) - *ReAct: Synergizing Reasoning and Acting in Language Models*
- Yao et al. (2023) - *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*
- Schick et al. (2023) - *Toolformer: Language Models Can Teach Themselves to Use Tools*
- Shinn et al. (2023) - *Reflexion: Language Agents with Verbal Reinforcement Learning*

Tools & Libraries

Category	Tools
Classical NLP	NLTK, spaCy, Gensim, scikit-learn
Deep Learning	PyTorch, TensorFlow, JAX
Transformers	Hugging Face Transformers, timm, torchvision
LLM Frameworks	LangChain, LlamaIndex, Haystack
Agents	LangGraph, CrewAI, AutoGen, Semantic Kernel, Anthropic Agent SDK, OpenAI Agents SDK
MCP	MCP Python SDK, MCP TypeScript SDK

Category	Tools
No-Code/Low-Code	n8n, Flowise, Dify, Zapier AI, Make.com
GraphRAG	Microsoft GraphRAG, Neo4j, LlamaIndex Knowledge Graphs
Vector DBs	Chroma, Pinecone, Weaviate, Qdrant, FAISS, Milvus
Embeddings	Sentence-Transformers, OpenAI, Cohere, Voyage AI
Inference	vLLM, TGI, Ollama, llama.cpp, LMStudio, LocalAI
Quantization	bitsandbytes, GPTQ, AWQ, GGUF
Evaluation	lm-evaluation-harness, Ragas, LangSmith, W&B, Phoenix
Fine-tuning	Hugging Face TRL, PEFT, Axolotl, Unsloth, OpenPipe
Observability	LangSmith, LangFuse, Weights & Biases, Arize Phoenix

Datasets for Practice

Classical NLP:

- IMDB Reviews (sentiment analysis)
- AG News, 20 Newsgroups (classification)
- CoNLL 2003 (NER)

Instruction Tuning & SFT:

- Alpaca, Dolly-15k, OpenAssistant
- ShareGPT, WizardLM, Evol-Instruct
- Domain-specific: MedInstruct, FinQA

Preference & Alignment:

- Anthropic HH-RLHF
- OpenAI WebGPT comparisons
- UltraFeedback, Capybara

RAG Documents:

- ArXiv papers, Wikipedia dumps
- PubMed, legal documents
- Custom enterprise documents (PDFs, Markdown)

Evaluation Benchmarks:

- MMLU, HellaSwag, ARC, TruthfulQA
- HumanEval, MBPP (coding)
- MT-Bench, AlpacaEval (instruction following)

Online Courses & Tutorials

Fundamentals:

- Stanford CS224N: NLP with Deep Learning
- Hugging Face NLP Course
- fast.ai: Practical Deep Learning

LLMs & Advanced Topics:

- deeplearning.ai LLM Courses
- LangChain Documentation
- LlamaIndex Guides

MCP & Agents:

- Model Context Protocol Documentation
- LangGraph Tutorials
- CrewAI Documentation

> [!TIP]
> **Post-Training Support:** Participants will receive access to a shared repository containing all lab notebooks, datasets, and additional resources for continued learning.